

Experiment No: 05

Name of the Experiment: Implementation of Bankers Algorithm using C++.

Introduction:

In a multi-programming environment, several processes may compete for limited system resources such as memory, printers and storage devices. If resources are allocated improperly, a situation called deadlock may occur, where processes wait indefinitely for resources held by one another. The Banker's Algorithm is a deadlock avoidance technique developed to ensure that the system remains in a safe state before granting resource requests. The algorithm checks whether allocating resources to a process will leave the system in a safe condition. If the system remains safe, the request is granted; otherwise, it is denied until resources become available. This experiment focuses on implementing the Banker's Algorithm using C++ to determine safe sequences and analyze resource allocation strategies. Understanding this algorithm helps in learning how operating systems prevent deadlocks and manage resources efficiently.

Objectives:

- To understand the concept of deadlock and resource allocation in operating systems.
- To learn the working principle of the Banker's Algorithm.
- To implement the Banker's Algorithm using C++.
- To calculate the need matrix from the allocation and maximum matrices.
- To determine whether the system is in a safe state.
- To find the safe sequence of process execution when one exists.
- To analyze how deadlock avoidance improves system reliability and efficiency.

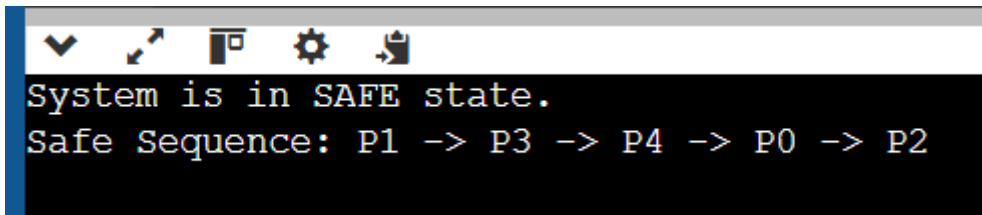
Algorithm:

1. Start the program.
2. Input the number of processes and resource types.
3. Input the Allocation matrix.
4. Input the Maximum matrix.
5. Input the Available resources vector.
6. Calculate the Need matrix using the formula: $\text{Need} = \text{Maximum} - \text{Allocation}$.
7. Initialize all processes as unfinished.
8. Search for a process whose Need is less than or equal to the Available resources.
9. If such a process is found, mark it as finished and add its allocated resources back to the Available resources.

10. Record the process in the safe sequence.
11. Repeat the search for the remaining unfinished processes.
12. Continue until all processes are completed or no further process can be executed safely.
13. If all processes finish successfully, display the safe sequence.
14. Otherwise, report that the system is in an unsafe state.
15. End the program.

Implemented Code:

<pre> #include <iostream> using namespace std; int main() { int n = 5; int m = 3; int alloc[5][3] = { {0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}}; int max[5][3] = { {7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}}; int avail[3] = {3, 3, 2}; int need[5][3]; for (int i = 0; i < n; i++) { for (int j = 0; j < m; j++) { need[i][j] = max[i][j] - alloc[i][j]; } } bool finish[5] = {false}; int safeSeq[5]; int count = 0; while (count < n) { bool found = false; for (int p = 0; p < n; p++) { </pre>	<pre> if (finish[p] == false) { bool possible = true; for (int j = 0; j < m; j++) { if (need[p][j] > avail[j]) { possible = false; break; } } if (possible) { for (int j = 0; j < m; j++) { avail[j] += alloc[p][j]; } safeSeq[count] = p; count++; finish[p] = true; found = true; } } } if (found == false) { cout << "System is NOT in safe state.\n"; return 0; } } cout << "System is in SAFE state.\n"; cout << "Safe Sequence: "; for (int i = 0; i < n; i++) { cout << "P" << safeSeq[i]; if (i != n - 1) cout << " -> "; } cout << endl; return 0; } </pre>
--	--

Output:A terminal window with a dark background and a light blue border. The window contains two lines of text in a monospaced font. The first line is "System is in SAFE state." and the second line is "Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2". Above the text, there is a toolbar with five icons: a checkmark, a cursor, a document, a gear, and a clipboard.

```
System is in SAFE state.  
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

Conclusion:

The implementation of the Banker's Algorithm demonstrates an effective method for avoiding deadlocks in operating systems. By evaluating resource requests before allocation, the algorithm ensures that the system remains in a safe state and prevents situations where processes become permanently blocked. The experiment highlights the importance of maintaining information about available, allocated and maximum required resources. It also shows how safe sequences can be determined to guarantee successful execution of all processes. Overall, the Banker's Algorithm provides a practical understanding of resource management and deadlock avoidance techniques, making it an essential concept in operating system design and implementation.

Experiment No: 10

Name of the Experiment: Implementation of VLAN in Cisco Packet Tracer II.

Introduction:

As network size grows, dividing devices into separate VLANs improves organization, security and traffic management. However, devices located in different VLANs cannot communicate directly because each VLAN acts as a separate logical network. To enable communication between VLANs, routing mechanisms must be introduced. Inter-VLAN routing allows devices from different departments or network segments to exchange data while maintaining VLAN separation. This experiment focuses on configuring VLANs and establishing communication between them in Cisco Packet Tracer.

Objectives:

- To create and configure multiple VLANs on a switch.
- To assign switch ports to their respective VLANs.
- To configure inter-VLAN routing for communication between different VLANs.
- To verify connectivity between devices located in separate VLANs.
- To analyze the role of routing in VLAN-based networks.

Theory:

A Virtual Local Area Network (VLAN) divides a physical network into multiple logical broadcast domains. Devices within the same VLAN can communicate directly through a switch, while communication between different VLANs is restricted by default. Inter-VLAN routing is required to allow data exchange across VLAN boundaries. This can be achieved using a router configured with multiple sub interfaces or a Layer 3 switch. Each VLAN is assigned a unique network address and gateway, enabling the routing device to forward packets between VLANs. This approach maintains network segmentation while allowing controlled communication when necessary. VLANs combined with routing provide improved scalability, security and traffic management in modern networks.

Lab work:

Network:

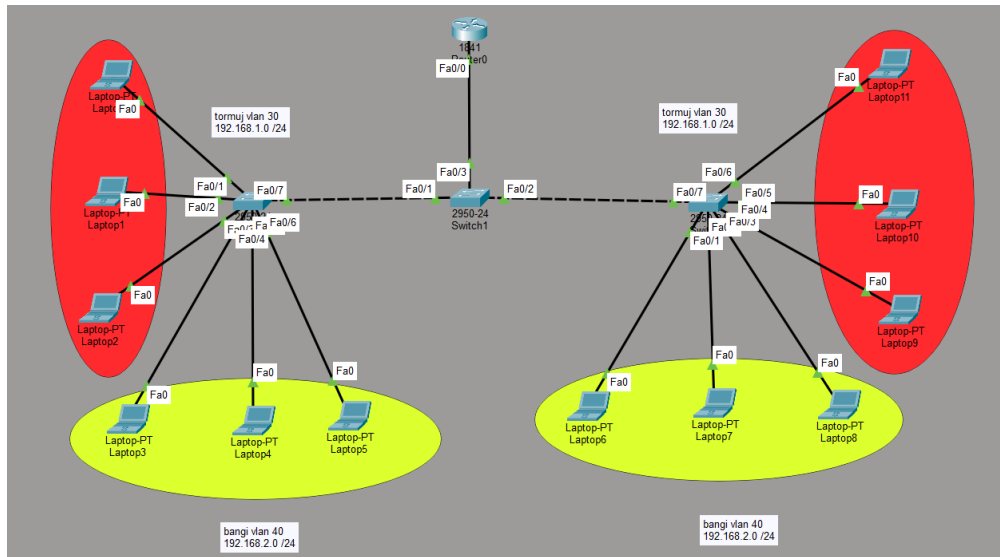


Fig-1: Implementation of VLAN using Two Different Switches in Cisco

Configuration:

```
Switch>en
Switch#config t
Enter configuration commands, one per line.
Switch(config)#vlan 30
Switch(config-vlan)#name tormuj
Switch(config-vlan)#exit
Switch(config)#vlan 40
Switch(config-vlan)#name bangi
Switch(config-vlan)#exit
```

Fig-2: VLAN Creation

```
Switch(config)#interface fa0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 30
Switch(config-if)#ex
Switch(config)#switchport mode access
Switch(config-if)#switchport access vlan 30

% Invalid input detected at '^' marker.

Switch(config)#interface fa0/2
Switch(config-if)#switchport mode access
Switch(config-if)#vlan 30
Switch(config-vlan)#ex
Switch(config)#interface fa0/3
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 30
```

Fig-3: VLAN “tormuj” configuration

```

Switch(config)#interface fa0/4
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 40
Switch(config-if)#ex
Switch(config)#interface fa0/5
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 40
Switch(config-if)#ex
^
% Invalid input detected at '^' marker.

Switch(config-if)#switchport access vlan 40
Switch(config-if)#exit
Switch(config)#interface fa0/6
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 40
Switch(config-if)#ex
^
% Invalid input detected at '^' marker.

Switch(config-if)#switchport access vlan 40
Switch(config-if)#ex

```

Fig-4: VLAN “bangi” configuration

```

Router#config t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fa0/0
Router(config-if)#no shut
Router(config-if)#interface fa0/0.30
Router(config-subif)#encapsulation dot1Q 30
Router(config-subif)#ip address 192.168.1.1 255.255.255.0
Router(config-subif)#ex
Router(config)#interface fa0/0.40
Router(config-subif)#encapsulation dot1Q 40
Router(config-subif)#ip address 192.168.2.1 255.255.255.0
Router(config-subif)#ex

```

Fig-5: Router configuration

Connection Test:

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Failed	Laptop1	Laptop7	ICMP		0.000	N	0	(edit)	
	Successful	Laptop1	Laptop7	ICMP		0.000	N	1	(edit)	

Fig-6: Message passing from Laptop1 to Laptop7

Conclusion:

The experiment successfully implemented multiple VLANs and established communication between them through inter-VLAN routing. Devices within separate VLANs were able to communicate after the routing configuration was completed. The results demonstrated how VLANs can enhance network management and security while still allowing controlled interaction between different network segments. This technique is widely used in enterprise networks to improve efficiency, scalability and administration.

Bangladesh Army University of Science and Technology (BAUST), Saidpur

Department of Computer Science and Engineering (CSE)



LAB REPORT

Department: Computer Science and Engineering

Course Title: Operating System Sessional

Course Code: CSE 3204

Report No: 05

Report Title: Implementation of Bankers Algorithm using C++.

Submitted By:

Name: Md. Iqbal Saklaen

ID: 0802320205101038

Level: 3 **Term:** II

Experiment Date: 20-05-2026

Submission Date: 07-06-2026

Submitted To:

Teacher Name:

Saad Ahmed

Nahid Anam Opu

Designation:

Lecturer

Lecturer

Signature:

Bangladesh Army University of Science and Technology (BAUST), Saidpur

Department of Computer Science and Engineering (CSE)



LAB REPORT

Department: Computer Science and Engineering

Course Title: Computer Networks Sessional

Course Code: CSE 3206

Report No: 10

Report Title: Implementation of VLAN in Cisco Packet Tracer II.

Submitted By:

Name: Md. Iqbal Saklaen

ID: 0802320205101038

Level: 3 **Term:** II

Experiment Date: 20-05-2026

Submission Date: 07-06-2026

Submitted To:

Teacher Name:

Md. Minhazul Islam

Iqramul Islam

Designation:

Lecturer

Lecturer

Signature:

